

Notes de soutenance

I Introduction

par William Fijean

II Abstraction

par William Fijean

III Traitement du fichier

A Conversion du fichier en commandes

1. On lit le fichier ligne par ligne (`fgets`),
2. Pour chaque ligne, on crée un `TokenSlice` en utilisant la fonction `strtok` pour découper la ligne en tokens (en utilisant les espaces comme séparateurs),
3. On crée un `CommandSlice` qui contient tous les `TokenSlice` du fichier, et on utilise la fonction `parseCommand` pour chaque `TokenSlice` afin d'exécuter les commandes correspondantes.

B Gestion des chemins

On utilise un slice de tableau de chaînes de caractères :

```
typedef struct chemin {  
    char *original; // chemin original  
    char **tokens; // tableau (dynamique) de sous-chaînes  
    int nbTokens; // nombre de sous-chaînes  
    int capacity; // capacité actuelle du tableau  
} Path;
```

IV Commandes

A cp, mv, touch/mkdir

par William Fijean

B pwd

Idée de l'algorithme :

1. On crée un slice de chaînes de caractères,
2. On remonte du noeud courant jusqu'à la racine, et à chaque fois on ajoute le nom du noeud courant en fin de slice,
3. On parcourt le slice à l'envers pour construire le chemin absolu comme suit :

```
1 result[0] = '\0';  
2  
3 if (iterator == 0)  
4 {  
5     strcpy(result, "/");  
6 }  
7 else
```

```

8   {
9       for (int i = iterator - 1; i >= 0; i--)
10      {
11          strcat(result, "/");
12          strcat(result, basenames[i]);
13      }
14  }

```

C rm

1. On cherche le noeud parent du noeud à supprimer (en utilisant la fonction `getParentFromPathIfExists`),
2. On cherche dans les fils directs du noeud parent celui qu'on veut supprimer,
3. Si on le trouve, on vérifie que ce n'est pas un parent du noeud courant (ou le noeud courant lui-même), sinon on affiche un message d'erreur,
4. Si c'est bon, on supprime le noeud (en utilisant la fonction `removeNode`).

D find

On ne regarde que le cas "classique" demandé par le sujet (`find NAME`) :

1. On parcourt récursivement l'arbre à partir du noeud de départ,
2. À chaque noeud, on vérifie si son nom contient le motif recherché (en utilisant la fonction `strstr`),
3. Si c'est le cas, on affiche le chemin absolu de ce noeud (en utilisant la commande `pwd`).

On a trois autres cas, sur lesquels on reviendra dans la partie *Bonus*.

V Bonus

A Arguments de find et automates

On utilise une fonction `match` qui prend en argument une chaîne de caractères et un motif, et qui retourne un booléen indiquant si la chaîne correspond au motif.

On a décidé de traiter trois cas, je ne présente que les deux qui sont opérationnels :

- `find *abc` : `strcmp(node->nom + (nameLength - (len - 1)), pattern + 1) == 0` (on vérifie que le motif est à la fin du nom),
- `find abc*` : `strncmp(node->nom, pattern, len - 1) == 0`

B Interpréteur de commandes shell

On a cette boucle infinie :

1. On stocke le caractère entré dans un `buffer` (dont on gère la taille),
2. Quand on reçoit un retour à la ligne, on crée un `TokenSlice` comme dans le cas de l'interpréteur fichier et on utilise `parseCommand`.

C Options de ls, help et printLine

par William Fijeau

VI Bugs

1. Certains textes sont en anglais (problème d'uniformisation),
2. Le `find *abc*` ne fonctionne pas (il cherche un facteur `abc*` avec `*` comme un caractère normal, et pas comme un joker).

Quelques pistes d'optimisation : insérer les fichiers en début de liste chaînée (on l'a fait par choix pour respecter les specs, mais ça casse la constance de la complexité de la commande `mkdir/touch`).

VII Annexes

A getParentFromPathIfExists

1. On vérifie que les arguments sont valides (non nuls),
2. On initialise un pointeur vers le noeud courant à partir du noeud de départ (qui peut être la racine ou le noeud courant selon que le chemin est absolu),
3. Si le chemin est vide, on retourne le noeud courant,
4. Sinon, on parcourt les tokens du chemin (sauf le dernier) :
 - (a) Si le token est `..`, on remonte au parent du noeud courant,
 - (b) Si le token est `.`, on reste sur le noeud courant,
 - (c) Sinon, on cherche parmi les fils du noeud courant un noeud dont le nom correspond au token. Si on le trouve, on descend dans ce noeud, sinon on retourne `NULL`.
5. Si on a parcouru tous les tokens sans problème, on retourne le noeud courant.
6. Complexité : $O(n \cdot m)$ où n est le nombre de tokens du chemin et m est le nombre de fils d'un noeud (dans le pire cas, on doit parcourir tous les fils pour trouver le token correspondant).

```

1  noeud *getParentFromPathIfExists(noeud *start, Path *path)
2  {
3      if (!start || !path)
4          return NULL;
5
6      noeud *current = start;
7      if (path->original[0] == '/')
8          current = current->racine;
9
10     if (path->nbTokens == 0)
11         return current;
12
13     for (int i = 0; i < path->nbTokens - 1; ++i)
14     {
15         if (strcmp(path->tokens[i], "..") == 0)
16         {
17             current = current->pere;
18         }
19         else if (strcmp(path->tokens[i], ".") == 0)
20         {
21             continue;
22         }
23         else
24         {
25             liste_noeud *next = current->fils;
26             int found = 0;
27             while (next)
28             {
29                 if (strcmp(next->no->nom, path->tokens[i]) == 0)
30                 {
31                     current = next->no;
32                     found = 1;
33                     break;
34                 }
35                 next = next->succ;
36             }
37             if (!found)
38                 return NULL;
39         }

```

```
40     }
41 }
42 return current;
43 }
```

B CommandSlice et TokenSlice

```
// Slice de tokens
typedef struct {
    char **content;
    int length;
    int capacity;
} TokenSlice;

// Slice des lignes
typedef struct {
    TokenSlice *lines;
    int length;
    int capacity;
} CommandSlice;
```